Desarrollo Web y Mobile - Proyecto de curso

Este es el planteo del proyecto de curso para el curso de *Desarrollo Web y Mobile* para el segundo semestre de 2026. Este documento está sujeto a cambios y correcciones.

1. Objetivo

Desarrollar una aplicación web y mobile que permita jugar al **Batalla Naval** y armar torneos entre varios jugadores.

2. Requerimientos funcionales

La aplicación completa cuenta con tres roles de *usuarios*: *Jugador*, *Organizador* y *Admin*.

2.1. Historias para todos los roles

- Como *Usuario* ingreso a la aplicación con mi correo electrónico y contraseña para poder usarla.
- Como *Usuario* cierro mi sesión, para que nadie más pueda usarla desde el navegador o la aplicación móvil.

2.2. Historias para *Jugadores*

- Como *Jugador* veo los torneos a los que he sido invitado.
- Como *Jugador* acepto una invitación a un torneo al que he sido invitado.
- Como *Jugador* rechazo una invitación a un torneo al que he sido invitado.
- Como *Jugador* veo las partidas en las que participo, terminadas o no.
- Como *Jugador* entro en una partida en curso, y veo los disparos de ambos jugadores con sus resultados.
- Como *Jugador* entro en una partida en curso, y si me toca hago una jugada.
- Como *Jugador* entro en una partida terminada, y veo el resultado (victoria o derrota) y el historial completo de jugadas.

2.3. Historias para *Organizadores*

- Como *Organizador* defino torneos con nombre.
- Como *Organizador* invito *jugadores* a participar de un torneo.
- Como *Organizador* revoco la invitación que hice a un *jugador* a participar de un torneo.
- Como *Organizador* armo las partidas de un torneo con los *jugadores* que aceptaron la invitación.
- Como *Organizador* defino una partida como abandonada por un *Jugador*, dejando al otro como ganador.
- Como *Organizador* defino el ganador de un torneo.
- Como *Organizador* puedo hacer todo lo que hace un *Jugador*, excepto jugar en los torneos que organizo.

2.4. Historias para *Admins*

- Como *Admin* agrego usuarios con su correo electrónico, contraseña y rol; así pueden ingresar a la aplicación.
- Como *Admin* modifico un usuario existente para cambiarle su contraseña o rol.
- Como *Admin* quito usuarios de la aplicación.
- Como *Admin* puedo hacer todo lo que hace un *Organizador*.

3. Otros requerimientos

La aplicación tendrá dos *frontends* implementados con Javascript: el *web* con *React*, y el *móvil* con *React Native* (para celulares Android). El diseño de las pantallas queda a discreción de los estudiantes.

Para mobile solo implementar las vistas y las funcionalidades del rol *Jugador*. Para web hay que implementarlas para todos los roles. Tener en consideración que si bien se pide una aplicación móvil, adicionalmente la interfaz web debe ser responsiva para soportar pantallas de dispositivos celulares.

Ambas interfaces de usuario (i.e. web y móvil) se comunicarán con una *API REST* provista por la cátedra. Se deberá utilizar JWT para la autenticación de usuarios.

La API no notifica a los clientes cuando el oponente hace una jugada. Para saber si es su turno, las interfaces deben consultar periódicamente la partida (`GET /api/matches/:id`) mientras el jugador la tenga abierta.

4. Entrega

La entrega del proyecto será el 18 de noviembre de 2026. Ese mismo día se estará tomando la defensa del proyecto.

4.1. Entregables

- Repositorio de GitHub con el código fuente de la aplicación web y móvil.
- Documentación del proyecto que incluya:
  - Instrucciones para ejecutar la aplicación.
  - Explicación de las funcionalidades implementadas.
  - Justificación de las decisiones técnicas tomadas.

4.2. Criterios de Evaluación

La evaluación del código entregado por los equipos se concentrará en:

- Implementación de las historias de usuario solicitadas.
- Código limpio y bien estructurado, con buenas prácticas de desarrollo.
- Diseño y usabilidad de las interfaces de usuario.

5. Modelo de datos

Todas las entidades manejadas por la API tienen un `id` autogenerado. Además se registran las fechas de creación y modificación de cada registro.

En el JSON de la API, todos los campos de tipo `Date` se envían como *strings* en formato ISO 8601 (e.g. `"2026-11-18T13:00:00.000Z"`).

```
interface Entity {
  id: string;
  createdAt: Date;
  modifiedAt: null | Date;
}
```

Los usuarios se identifican por el *correo electrónico* con el que se registraron a la aplicación. Ingresan con una *clave*, y su sesión actual está identificada por un `token` (JWT).

```
type UserRole = 'player' | 'organiser' | 'admin';

interface User extends Entity {
  email: string;
  password: string;
  role: UserRole;
}

interface Login {
  email: User['email'];
  role: User['role'];
  token: string;
  userId: User['id'];
}
```

`Board` es la representación de un tablero, dando una *altura* (cantidad de filas) y un *ancho* (cantidad de columnas), y los *barcos* que se pueden ubicar (cada uno con un nombre y largo en casillas).

```
interface Board {
  height: number;
  width: number;
  ships: Record<string, number>;
}
```

Las partidas siempre están dentro de un *torneo*. Todos los torneos los crean usuarios organizadores (o admins), que además son los que deciden cuál jugador gana el torneo.

```
interface Tournament extends Entity {
  name: string;
  organisedBy: User['id'];
  winner: User['id'] | null;
}
```

Los jugadores tienen que ser invitados a un torneo para poder participar. Las invitaciones son hechas por los usuarios organizadores, y pueden ser aceptadas o rechazadas por los usuarios jugadores.

```
interface Invitation extends Entity {
  invitedUser: User['id'];
  invitingUser: User['id'];
  tournament: Tournament['id'];
  acceptedAt: Date | null;
  rejectedAt: Date | null;
}
```

`Match` es la representación de una partida. Para hacer el primer movimiento (poner barcos en el tablero) un jugador no tiene que esperar por el otro. A partir de que ambos jugadores ponen sus barcos, ahí sí el juego alterna turnos. `playerNorth` es siempre el primer jugador en disparar, mientras que `playerSouth` es siempre el segundo.

`fleetNorth` y `fleetSouth` son las distribuciones de los barcos en el tablero de cada jugador. Previo a que los jugadores hagan su primer movimiento son `null`. Previo a que la partida termine, la flota de cada jugador aparece como `null` para su oponente. Terminada la partida ambos jugadores pueden ver ambas flotas.

Los campos `shotsNorth` y `shotsSouth` son los registros de los disparos hechos por ambos jugadores, y sus correspondientes resultados. `shotsNorth` tiene, en orden, los disparos de `playerNorth` sobre el tablero de `playerSouth`, y `shotsSouth` los de `playerSouth` sobre el tablero de `playerNorth`.

`BoardCoordinate` representa una coordenada en el tablero, con columna y fila, ambos comenzando en 0. La coordenada `[0, 0]` es la esquina superior izquierda, y `[width - 1, height - 1]` la inferior derecha.

Una ubicación de barcos (`FleetSetup`) es válida si incluye todos los barcos definidos en `Board.ships` y ningún otro. Cada barco lista todas las casillas que ocupa, tantas como su largo, contiguas y en línea recta (horizontal o vertical), y dentro del tablero. Los barcos no pueden superponerse, pero sí pueden estar en casillas adyacentes.

Un barco se hunde (`'sunk'`) cuando todas sus casillas recibieron un disparo. La partida termina cuando un jugador hunde todos los barcos de su oponente, quedando como ganador. También puede terminar si un organizador la da por terminada (ver `POST /api/matches/:id/end`).

```ts
type Player = 'north' | 'south';
type BoardCoordinate = [number, number]; // [column, row]
type FleetSetup = Record<string, BoardCoordinate[]>;
type ShotResult = 'water' | 'hit' | 'sunk';

interface Match extends Entity {
  tournament: Tournament['id'];
  playerNorth: User['id'];
  playerSouth: User['id'];
  endedAt: Date | null;
  winner: Player | null;
  fleetNorth: FleetSetup | null;
  fleetSouth: FleetSetup | null;
  shotsNorth: [BoardCoordinate, ShotResult][];
  shotsSouth: [BoardCoordinate, ShotResult][];
}
```

`MatchMove` es la representación de una acción en la partida. El primer movimiento de ambos jugadores en todas las partidas siempre es un `FleetSetup`, es decir una ubicación de barcos en el tablero de cada jugador. Luego de estas acciones en adelante, cada movimiento es la coordenada de un disparo.

Los movimientos se hacen con `POST /api/matches/:id/moves`: el cuerpo `{ fleetSetup }` genera un movimiento con `move` de tipo `FleetSetup`, y el cuerpo `{ shot }` uno con `move` de tipo `BoardCoordinate`. El historial de movimientos solo está disponible cuando la partida terminó. Mientras está en curso, los disparos se pueden ver en `shotsNorth` y `shotsSouth` de la partida.

```ts
interface MatchMove extends Entity {
  match: Match['id'];
  index: number;
  player: Player;
  move: FleetSetup | BoardCoordinate;
}
```

6. API REST

Tanto los cuerpos de peticiones como los de las respuestas usarán JSON como formato. Todas las rutas excepto `PUT /api/me/session`, requieren el token de sesión en el *header* `Authorization` (de la forma `Bearer <token>`).

6.1. Todos los usuarios

- `GET /api/me`: Obtener información del usuario actual.
  - Respuesta OK 200 `Omit<User, 'password'>`
- `PUT /api/me/session`: Ingreso de un usuario a la aplicación, retornando un *token* de sesión.
  - Cuerpo `{ email: string, password: string }`
  - Respuesta OK 200 `Login`
- `DELETE /api/me/session`: Cierre de sesión del usuario actual.
  - Respuesta OK 204

6.2. Jugadores

- `GET /api/board`: Obtiene la definición del tablero.
  - Respuesta OK 200 `Board`
- `GET /api/invitations/:id`: Obtiene una invitación por id.
  - Respuesta OK 200 `Invitation`
- `POST /api/invitations/:id/accept`: Aceptar una invitación.
  - Respuesta OK 200 `Invitation`
- `POST /api/invitations/:id/reject`: Rechazar una invitación.
  - Respuesta OK 200 `Invitation`
- `GET /api/matches/:id`: Obtener una partida por id del usuario actual.
  - Respuesta OK 200 `Match`
- `GET /api/matches/:id/moves`: Obtener la historia de jugadas de una partida por id, si está finalizada.
  - Respuesta OK 200 `MatchMove[]`
- `POST /api/matches/:id/moves`: Hacer una jugada en una partida dada por id.
  - Cuerpo `{ fleetSetup: FleetSetup } | { shot: BoardCoordinate }`
  - Respuesta OK 200 `Match`
- `GET /api/me/invitations`: Listar las invitaciones del usuario actual.
  - Respuesta OK 200 `Invitation[]`
- `GET /api/me/matches`: Listar las partidas del usuario actual.
  - Respuesta OK 200 `Match[]`
- `GET /api/me/tournaments`: Listar torneos en los cuales participa el usuario actual (invitaciones aceptadas).
  - Respuesta OK 200 `Tournament[]`
- `GET /api/tournaments/:id`: Obtener un torneo por id.
  - Respuesta OK 200 `Tournament`
- `GET /api/tournaments/:id/matches`: Listar partidas de un torneo por id.
  - Respuesta OK 200 `Match[]`

6.3. Organizadores

- `PUT /api/invitations`: Invitar un usuario a un torneo.
  - Cuerpo `{ invitedUser: User['id'], tournament: Tournament['id'] }`
  - Respuesta OK 201 `Invitation`
- `DELETE /api/invitations/:id`: Eliminar una invitación.
  - Respuesta OK 204
- `DELETE /api/matches/:id`: Borra una partida por id.
  - Respuesta OK 204
- `POST /api/matches/:id/end`: Fuerza el fin de una partida, dando a uno de los jugadores como ganador.
  - Cuerpo `{ winner: Player }`
  - Respuesta OK 200 `Match`
- `PUT /api/tournaments`: Crear un torneo.
  - Cuerpo `{ name: Tournament['name'] }`

  - Respuesta OK 201 `Tournament`
- `PATCH /api/tournaments/:id`: Modificar un torneo.
  - Cuerpo `{ name?: string, winner?: User['id']; }`
  - Respuesta OK 200 `Tournament`
- `DELETE /api/tournaments/:id`: Eliminar un torneo.
  - Respuesta OK 204
- `GET /api/tournaments/:id/invitations`: Lista las invitaciones a un torneo por id.
  - Respuesta OK 200 `Invitation[]`
- `PUT /api/tournaments/:id/matches`: Crear una partida en un torneo por id.
  - Cuerpo `{ playerNorth: User['id'], playerSouth: User['id'] }`
  - Respuesta OK 201 `Match`
- `GET /api/users`: Listar todos los usuarios (sin contraseñas).
  - Respuesta OK 200 `Omit<User, 'password'>[]`

6.4. Admins

- `PUT /api/users`: Crear un usuario.
  - Cuerpo `{ email: string, password: string, role: UserRole }`
  - Respuesta OK 201 `Omit<User, 'password'>`
- `DELETE /api/users/:id`: Eliminar un usuario.
  - Respuesta OK 204
- `GET /api/users/:id`: Obtener información de un usuario específico (sin contraseña).
  - Respuesta OK 200 `Omit<User, 'password'>`
- `PATCH /api/users/:id`: Modificar un usuario.
  - Cuerpo `{ password?: User['password'], role?: User['role'] }`
  - Respuesta OK 200 `Omit<User, 'password'>`

6.5. Aspectos comunes

- Todos los *endpoints* requieren un encabezado `x-api-key` con un identificador de base de datos provisto por la cátedra. De no estar o no tener un valor válido, la respuesta es un error HTTP 404 sin contenido. Este control se hace antes que cualquier otro.
- Todos los *endpoints* menos `PUT /api/me/session`, requieren que el usuario haya iniciado sesión y pase el token en el encabezado de `Authorization` de la petición HTTP. Si este encabezado falta, o la sesión no es válida, la API retorna un error HTTP 401 sin contenido.
- Si el usuario pasa un token de sesión válido, pero su rol no le permite llamar al *endpoint*, la API retorna un error HTTP 403 sin contenido.
- Si el `id` de la ruta no corresponde a un registro existente, o a uno que el usuario puede ver (e.g. una partida de otros jugadores, para un *Jugador*), la API retorna un error HTTP 404 sin contenido.
- Todas las veces que la API retorna un error HTTP 400, el cuerpo de la respuesta tendrá la forma `{ error: string }`. Esto sucede si se intenta:
  - Iniciar sesión con un `email` o `password` no válidos.
  - Aceptar o rechazar una invitación que ya está aceptada o rechazada.
  - Invitar a una persona que ya está invitada al torneo, y esta invitación no está rechazada.
  - Invitar al organizador de un torneo a participar de ese mismo torneo.
  - Borrar una invitación a un torneo de un usuario que ha participado en partidas de ese torneo.
  - Crear una partida con un usuario que no aceptó una invitación al torneo de la partida.
  - Crear una partida con el mismo usuario en ambos roles.
  - Hacer una jugada en una partida que está terminada (i.e. tiene ganador).
  - Hacer una jugada con un jugador cuando no es su turno.
  - Hacer una jugada que no es válida (e.g. disparar a una casilla a la que ya se disparó, o fuera del tablero).
  - Ubicar barcos de forma no válida (ver sección 5), o ubicarlos por segunda vez en la misma partida.

- Disparar antes de que ambos jugadores hayan ubicado sus barcos.
- Obtener el historial de jugadas de una partida que no está terminada.
- Poner como ganador de una partida a un jugador que no es uno de los participantes.
- Declarar como ganador de un torneo a un usuario que no ha ganado ninguna partida del torneo.
- Borrar un usuario que ha participado de alguna partida.

Fin